

Personal Cybersecurity Guardian Agent

Project Narrative

Anusha Prakash | Student ID: 002306070

INFO7375 - Prompt Engineering & AI | Northeastern University

April 24, 2026

1. Where the Idea Came From

The idea did not start with architecture. It started with a specific frustration that built up over four years of enterprise security consulting.

At Simeio, I spent four years deploying BeyondTrust, Oracle Identity Manager, and Active Directory for large organizations. I worked with VirusTotal, reviewed CVSS scores, and navigated the NIST NVD regularly. These are genuinely powerful tools - accurate, well-maintained, and comprehensive. But they are built entirely for practitioners. Every interface assumes the user already knows what CVSS 9.8 means, what a CPE string is, and why CVE-2021-44228 should make them act immediately.

The gap I kept thinking about was not a technical gap. It was a translation gap. The FBI documented 12.5 billion dollars in cybercrime losses in 2023, with phishing the leading category. The people paying that cost are not corporations with security teams they are individuals, small business owners, students, people who use the internet every day and have no tool designed for them. The intelligence exists. CISA publishes a daily-updated catalog of every actively exploited vulnerability. VirusTotal scans any URL against 70+ engines in seconds. PhishTank maintains 57,000+ verified phishing URLs. None of this is accessible to the person who just received a suspicious email and wants a simple answer.

The thesis became: what if a non-technical person could paste a suspicious URL or type 'I have Log4j 2.14.1 installed' and receive a plain-English verdict with a reason and a next step? Not a CVSS vector. Not an engine count. A sentence. Enough to act on.

That single question drove every architecture decision that followed.

2. Proposal Evolution: v1 to v3

2.1 What v1 Got Right and Wrong

The v1 proposal named the right problem but left the solution underspecified. It described two agents and a synthesis layer, but the synthesis layer said only that both agents would investigate independently. That describes a parallel process. It says nothing about what happens when the two agents disagree, which is the exact case that determines whether the system is trustworthy.

FLAN-T5 fine-tuning appeared as an architectural component rather than a hypothesis. ChromaDB and Pinecone were both referenced without resolution. These were not stylistic gaps they were decisions that had been deferred with language that sounded like decisions.

2.2 The Gru Review - Four Structural Gaps

Before finalizing v2, the proposal was submitted to Gru (Software Design Document Expert, Professor Nik Bear Brown's tool directory). Gru identified four specific gaps that would have produced failures during implementation.

Gap	Why It Mattered	Fix Applied in v3
-----	-----------------	-------------------

Synthesis layer: 'both agents investigate independently' - no conflict resolution	No specification means the implementation makes the decision. Implementation-level synthesis choices produce inconsistent verdicts.	Five explicit conflict resolution rules. Rule 1: MALICIOUS surfaces regardless. Rule 2: VULNERABLE escalates over SAFE. Rules 3-5 cover INSUFFICIENT_EVIDENCE and confidence thresholds.
Router mentioned once, never specified - no routing conditions defined	An unspecified router is a placeholder. Someone fills it in during coding, making architectural decisions without architectural review.	Full routing table covering five input scenarios including ambiguous input and incomplete version strings.
FLAN-T5 framed as component, not hypothesis	An architecture that cannot be falsified cannot be evaluated.	Reframed as testable hypothesis with RAGAS faithfulness threshold. GPT-4o mini named as fallback path.
ChromaDB and Pinecone both referenced without resolution	Two vector database references with no specification is a dependency conflict.	Pinecone-only across all environments. ChromaDB removed entirely.

2.3 The Professor's Addition - The Fourth Component

Between v2 and v3, the course professor provided an abstract introducing the Contextual Intervention Layer described as 'the difference between a system that informs and a system that protects.'

This was architecturally significant. The original synthesis layer produced a verdict and a summary. The intervention layer produces a situation-specific action plan different templates for someone who has not yet clicked a malicious link versus someone who already has, for someone whose software is vulnerable versus someone whose query is inconclusive. The correct next step for a person who has already clicked a phishing link is fundamentally different from advice for someone who has not.

Recognizing this as a distinct fourth component rather than treating it as a rephrasing of the summary output was the right call and one, I am glad the professor pushed.

3. Professor's Tools - How Each Was Used

3.1 Gru - Software Design Document Expert

Used during proposal review (v2 to v3). The four gaps Gru identified are documented in Section 2.2. Every one of them produced a concrete v3 change.

At project completion, the `/claude` command was run to generate a boondoggle score a structured assessment of which parts of the build were executed by Claude versus decided by the human.

Gru Boondoggle Score: 87/100 — Strong. Plausibility auditing 19/20. Problem formulation 17/20. Tool orchestration 18/20. Interpretive judgment 17/20. Executive integration 16/20. Labor split: Claude ~72% of lines

and tokens produced. Human ~28% of decisions that mattered. Score interpretation: 'The problem is not that you typed less. The problem would have been if the 28% had been the wrong 28%. It wasn't.'

3.2 CRITIQ - Peer Review and Paper Development Protocol

Used after completing the Technical Report and Validation Report. CRITIQ reviewed both documents with the rigor of a journal peer reviewer and produced ranked findings.

The Technical Report review: six findings (one Critical, two Major, three Minor). The Critical finding correctly identified that mock-mode evaluation results (100% precision and recall) were anchoring the Abstract and Conclusion without equal weight given to the live evaluation results. The revision separated mock-mode results (pipeline logic validation) from live VirusTotal results (external detection validation) throughout the document.

The Validation Report review: seven findings, addressing the PhishTank PASS label in the summary table (relabelled to OPERATIONAL CONFIRMED), the VirusTotal timeout rate as an operational finding, and the anti-hallucination sample composition.

A point-by-point response was drafted for both reviews. The response pushed back on one characterization in the Technical Report review the claim that the mock evaluation was tautological while conceding all other findings. The correct framing: mock evaluation tests the full pipeline (routing, synthesis, conflict resolution, intervention), not just the mock engine layer. Both revised documents include a Review History section documenting every change made in response to CRITIQ findings.

3.3 Bacon - Academic Writing Coach

Used after completing the Technical Report Final version. Bacon ran a diagnostic scan and returned a paper readiness score of 4/5, rising to 4.5/5 after revisions.

The most actionable findings: APA in-text citations were entirely absent throughout the body - the references section existed but no inline (Author, Year) calls appeared anywhere. Bacon identified eight citation gaps including RAG accuracy, sentence transformer retrieval quality, and LLM cybersecurity application literature. All were addressed in the final version with peer-reviewed citations added to the Related Work section.

Bacon also confirmed the unexecuted baseline comparison as the largest structural gap between the thesis framing and the evidence. This prompted the addition of Section 6.8 the baseline comparison between Guardian Agent and GPT-4o mini without retrieval augmentation. The paper readiness score increased from 4/5 to 4.5/5 following these revisions.

3.4 Brutalist - Presentation Design

Used to produce the final course presentation deck. The /deck silent command was run with the Technical Report as input. Brutalist generated a 16-slide deployable HTML presentation using the Musinique brutalist design system a single self-contained file navigable by keyboard arrow keys.

The deck covers: problem statement with the FBI \$12.5B cybercrime figure, system architecture with the four-component pipeline, data sources with indexed record counts, the baseline comparison result, evaluation results (100% precision, 85% Recall@5, 20/20 conflict resolution), implementation challenges,

known limitations, three-tier future roadmap, and a red conclusion slide anchored by the three key metrics. Every slide title is a claim, not a topic - Brutalist's core design principle. The deck is deployed alongside the project as a standalone HTML file.

4. What the System Actually Does

The Personal Cybersecurity Guardian Agent is a four-component multi-agent AI system deployed publicly at <https://huggingface.co/spaces/prakashanusha/cybersecurity-guardian>.

A non-technical user pastes a suspicious URL, email text, or software version. The system returns a plain-English verdict MALICIOUS, VULNERABLE, SAFE, UNCERTAIN, or CLARIFY with cited evidence, a risk score from 0 to 100, and a situation-specific action plan.

What 'cited evidence' means: *Every verdict includes the live sources that produced it. For a phishing verdict: 'VirusTotal scan: 25/72 engines flagged. Domain registered 3 days ago.' For a vulnerability verdict: 'CVE-2021-44228, CVSS 10.0 CRITICAL confirmed in CISA Known Exploited Vulnerabilities catalog.' These claims are independently verifiable at [virustotal.com](https://www.virustotal.com), nvd.nist.gov, and cisa.gov. A base LLM cannot provide this.*

The system indexes 149,831 threat intelligence records across two Pinecone namespaces: 92,126 CVEs from CISA KEV and NIST NVD (CVSS 7.0 or above), and 57,705 verified phishing patterns from PhishTank. These records are retrieved in real time for every query not cached in model weights with a training cutoff date.

5. Key Decisions That Were Mine

The Gru boondoggle framework distinguishes between what Claude executes and what only a human can decide. The following decisions were mine. Claude had no information that would have produced them.

5.1 Pinecone-Only Architecture

The original proposal referenced both ChromaDB (local) and Pinecone (production) with a Week 4 migration step. Before writing a single line of indexing code, I identified the problem: Render.com's free tier uses an ephemeral filesystem. Every cold start wipes a local index, requiring a complete re-index of 90,000 records before the application can respond to any query. This is a deployment-killing architectural flaw that only becomes visible in production.

Decision: Pinecone-only across all environments from Day 1. One index, two namespaces, no migration step. Claude did not flag the Render.com constraint. I identified it by reasoning about how the deployment environment actually works.

5.2 NVD Download Decision

The NVD download ran for three hours and captured 179,281 records before timing out. The instinct was to resume. Instead, I checked the year distribution: the downloaded records covered 1988 to 2022 entirely. The missing records were 2023 to 2025 but CISA KEV covers every actively exploited CVE from that period. The decision: stop, apply CVSS 7.0 or above filter, reduce to 90,560 records. Low-severity CVEs are not actionable for a non-technical user. This was three distinct judgments made in sequence. Claude ran the year-distribution script. The decision about what the analysis meant was not a script.

5.3 Two-Pass Search Architecture

After adding NVD records to the Pinecone index, Recall@5 dropped from 85 percent to 65 percent. NVD records were outranking KEV records for queries about actively exploited software. The surface fix was to add a known_exploited filter. The architectural insight was that actively exploited CVEs should always surface before theoretical ones pass one filters KEV (urgent, confirmed exploited), pass two performs broader NVD search. This architecture was not in the original design. It emerged from a retrieval failure and became better architecture.

5.4 FLAN-T5 Decision

FLAN-T5 fine-tuning was designed to translate technical CVE descriptions into plain English. After evaluating GPT-4o mini's synthesis prompt output quality, it was clear the threshold was already being met without fine-tuning. The decision to drop FLAN-T5 required evaluating the synthesis outputs against the purpose the fine-tuning was supposed to serve which required knowing what 'good enough' meant for this use case.

5.5 Recognizing the Professor's Addition as Architecturally Distinct

When the professor's abstract introduced the Contextual Intervention Layer, the decision was whether this was a new component or a rephrasing of the synthesis summary. It is a new component: it uses different prompt templates, produces different output for different user contexts (clicked vs. not clicked), and makes no retrieval calls. Treating it as a paraphrase would have produced a weaker system.

6. Challenges and Resolutions

6.1 PhishTank Metadata Size Constraint

PhishTank indexing failed at 90 percent completion with a Pinecone metadata size error 51,290 bytes exceeding the 40,960 byte limit. Some phishing URLs contain session tokens and tracking strings that make them extremely long. The resolution required understanding the architecture: VirusTotal receives the full URL directly from the user; Pinecone stores pattern metadata for semantic similarity. Truncating the URL in metadata to 300 characters has no effect on search quality because the embedding is generated from full text before truncation.

6.2 FastAPI Event Loop Conflict

FastAPI runs an asyncio event loop. The phishing agent's VirusTotal call is also asynchronous. Nesting asyncio.run() inside an already-running loop raises RuntimeError in Python 3.10 and later. nest_asyncio

was tried first it resolved the immediate error but introduced a `TypeError` with `uvicorn`'s `loop_factory` in Python 3.13. The resolution was a `ThreadPoolExecutor` with a fresh event loop per thread, completely isolated from FastAPI's event loop.

6.3 PhishTank Recall Metric Invalidity

The original evaluation used `Recall@5` by brand tag the same metric used for CVE retrieval. This returned 50 percent on what were clearly correct retrievals. The diagnosis: 93 percent of PhishTank records carry an 'Other' brand tag. `Recall@5` was measuring PhishTank's metadata quality, not retrieval quality. The evaluation was redesigned to measure end-to-end verdict accuracy across 40 labeled URLs a better metric for the actual question being asked.

6.4 HF Spaces Input Persistence

On Hugging Face Spaces, the text input cleared on every Analyze button click due to Streamlit's page rerun behavior. This did not occur during local development. The fix was `st.session_state` to persist the input value across reruns a one-function change that resolved the issue.

7. Human-AI Collaboration - The Honest Account

7.1 What Claude Executed

Claude wrote all Python code router, agents, synthesis, intervention, API, and Streamlit frontend. Claude wrote all indexing scripts, all test suites, and all document drafts including the proposals, technical report, validation report, and this narrative. Claude debugged every error traceback and generated the architecture diagram.

Without Claude, this project would have taken months, not days. That is not an exaggeration.

7.2 What Required Human Judgment

The quality of what Claude built is a direct function of the quality of what I specified. When I specified poorly as in the v1 proposal Claude built something incomplete. When I specified precisely as in the v3 synthesis layer with five explicit conflict resolution rules Claude built something that passed 20/20 test scenarios.

Every decision that mattered required domain knowledge, interpretive judgment, or accountability that cannot be delegated:

- Pinecone-only decision required knowing how Render.com's free tier works in production.
- NVD download decision required understanding what CISA KEV covers and what a non-technical user needs.
- Two-pass search design required distinguishing a test failure from an architecture signal.
- FLAN-T5 removal required evaluating output quality against a standard that was not in the code.
- Baseline comparison framing required deciding what evidence would actually be valid versus what would look good.

- CRITIQ response required understanding the difference between the mock engine's behavior and the full pipeline's behavior.

Gru on the labor split: *'Plausibility auditing is the hardest capacity to demonstrate because it requires you to notice a problem before it produces evidence of itself. The Render.com catch, the Recall@5 reframe, the VirusTotal stop these are not debugging. These are the skill of hearing the wrong note before the orchestra finishes the measure.'*

7.3 The Right Framing

Claude solves faster than any human and that gap will not close. What will not change: Claude cannot verify whether its output is grounded in specific domain reality, cannot reframe a poorly formulated problem, and cannot integrate multiple legitimate but conflicting perspectives into a recommendation that someone is accountable for.

The 87/100 boondoggle score is a record of which parts of this project required the capacities that cannot be automated. The score is not about how much I typed. It is about whether the 28 percent of decisions that were mine were the right 28 percent.

8. What the Project Demonstrates

8.1 The Core Thesis - Confirmed

A baseline comparison between the Guardian Agent and GPT-4o mini without retrieval augmentation was conducted on five inputs covering the full routing and verdict range. Both systems achieved equivalent verdict accuracy on well-known historical threats. The Guardian Agent's differentiation is in three dimensions:

Dimension	GPT-4o mini (no retrieval)	Guardian Agent
Verdict accuracy (known threats)	5/5 correct	5/5 correct
Evidence source	Training memory - fixed cutoff date	Live APIs - real-time, verifiable
CVE citations	From training memory - unverifiable	From live NVD - verify at nvd.nist.gov
Confidence score	None	Structured 0-100% per verdict
Calibrated uncertainty	General advice without confidence grounding	UNCERTAIN when evidence is mixed
Version enforcement	Gives partial answer without version	Requires full version, refuses to speculate

Live source citations	None	VirusTotal, NVD, CISA KEV per verdict
-----------------------	------	---------------------------------------

The differentiation that matters most for the target user a non-technical person asking about a threat they encountered today is evidence verifiability. When the Guardian Agent says a URL has 25/72 engines flagged and was registered 3 days ago, that claim is verifiable. When GPT-4o mini says a URL looks suspicious, that claim is not.

8.2 The Design Property That Matters Most

The system's most important safety property is calibrated uncertainty. When VirusTotal returns zero engine flags for an unknown URL, the system returns UNCERTAIN with supporting Pinecone pattern evidence not SAFE. When a software version query retrieves no CVE records, the system explicitly notes that absence of a record does not guarantee safety.

This is not a fallback. This is the behavior that makes the system safe for a non-technical user. A system that returns SAFE without evidence is more dangerous than one that says it does not know. Every evaluation result and every architecture decision in this project has been oriented around that principle.

8.3 Honest Scope

The system is a proof of concept. It uses a snapshot of threat intelligence current as of April 2026, deployed on a free Hugging Face Space with VirusTotal's 500-request daily limit. The gap between this proof of concept and a production system is documented continuous KEV sync, Redis caching, browser extension, OS-aware interventions. These are not speculative features. They are the architectural decisions that separate a demo from a product. Documenting them honestly is part of what makes a proof of concept credible.

9. Professor's Tools - Summary

Tool	Stage	What It Produced	Impact
Gru - SDD Expert	Proposal v2 review	4 structural gaps identified	Led directly to v3: synthesis rules, router specification, FLAN-T5 reframing, ChromaDB removal
Gru Boondoggle Score	Project completion	87/100 - Strong	Documents human contribution across 5 supervisory capacities
CRITIQ	Technical Report review	6 findings (1 Critical, 2 Major, 3 Minor)	Mock/live evaluation distinction, rate-limit operational finding, thesis framing
CRITIQ	Validation Report review	7 findings (1 Critical, 3 Major, 3 Minor)	PhishTank label correction, timeout finding, anti-hallucination sample description

Bacon	Technical Report Final - diagnostic scan	4/5 → 4.5/5 after revisions	APA citations added throughout, baseline comparison section prompted and added
Brutalist	Final demo presentation	16-slide HTML deck (Musinique brutalist design)	Complete - covers problem, architecture, evaluation, challenges, limitations, future work, conclusion

Appendix: Gru Boondoggle Score - Full Results

The following is the complete boondoggle score produced by running the completed project through Gru's /claude command.

Overall: 87 / 100 — Strong. Build labor split: Claude ~72% of lines/tokens produced. Human ~28% of decisions that mattered.

Capacity	Score	Evidence cited by Gru
1. Plausibility auditing	19/20	Caught Render.com ephemeral filesystem before deployment. Recognized 65% Recall@5 drop as architecture signal, not test failure. Recognized FLAN-T5 was adding complexity without adding accuracy. Stopped VirusTotal run when first-run data was sufficient.
2. Problem formulation	17/20	Reframed 40-URL recall test when results were low - understood what the data actually meant. Identified the professor's fourth component as architecturally distinct, not paraphrase. Stopped NVD download at 179K with deliberate filter logic. Chose not to run post-hoc baseline - correctly formulated what evidence would actually be valid.
3. Tool orchestration	18/20	Pinecone-only decision sequenced correctly - infrastructure constraint resolved before code written. CVSS >= 7.0 filter applied before ingestion. Two-pass search design adopted as architecture response to recall data. HF Spaces deployment decision made before build committed to split infrastructure path.
4. Interpretive judgment	17/20	Evaluated Gru's four critiques and distinguished substantive from stylistic. Interpreted live URL recall as expected operational behavior, not system failure. GPT-4o mini synthesis adequacy judgment required domain-contextual reading of output quality.
5. Executive integration	16/20	UI tab layout decision integrated UX, deployment, and scope constraints simultaneously. FLAN-T5 removal integrated across evaluation framework, cost model, and architecture. The build reached a deployable state with coherent architecture across all components.

Score interpretation (Gru): 'Not how hard you worked. Not how much you typed. How precisely you conducted the build - which decisions you owned, which you deferred, and whether the deferral was deliberate.'